

IC 設計流程 AI Agent 整合

挑戰分析與待辦事項報告

2026 年 4 月

1. 執行摘要.....	2
2. 工具整合與介面標準化.....	3
2.1 當前困境	3
2.2 新興解決方案	3
2.3 待辦事項	3
3. LLM 的技術限制與可靠性	4
3.1 幻覺與程式碼生成問題	4
3.2 推理與擴展性挑戰.....	4
3.3 待辦事項	4
4. 設計資料與訓練數據	5
4.1 數據稀缺的惡性循環	5
4.2 多模態資料處理	5
4.3 待辦事項	5
5. 驗證與正確性保證	6
5.1 零容忍需求	6
5.2 當前挑戰	6
5.3 待辦事項	6
6. 安全性與 IP 保護	7
6.1 核心風險	7
6.2 待辦事項	7
7. 基礎設施與運算資源	8
7.1 運算需求	8
7.2 待辦事項	8
8. ECO Friendly Design（工程變更友善設計）	9
8.1 什麼是 ECO Friendly Design.....	9
8.2 AI Agent 在 ECO 流程中的挑戰	9
8.3 新興解決方案	9
8.4 待辦事項	9
9. 人機協作與組織變革	11
8.1 人在迴圈的必要性.....	11
9.2 待辦事項	11
10. 待辦事項總表	12
10.1 高優先級（建議優先執行）	12
10.2 中優先級	12
10.3 低優先級	13
11. 參考資料.....	14

1. 執行摘要

隨著大型語言模型（LLM）與 Agentic AI 技術的快速發展，半導體產業正積極探索將 AI Agent 整合至 IC 設計流程中。然而，要建立一個能夠端到端自主執行 IC 設計任務的 AI Agent 系統，尚面臨諸多技術與工程挑戰。

本報告綜合多篇學術論文與產業報告，彙整主要困難點並列出需要完成的具體事項，涵蓋以下七大領域：

- 工具整合與介面標準化
- LLM 的技術限制與可靠性
- 設計資料與訓練數據
- 驗證與正確性保證
- 安全性與 IP 保護
- 基礎設施與運算資源
- ECO Friendly Design（工程變更友善設計）
- 人機協作與組織變革

2. 工具整合與介面標準化

2.1 當前困境

IC 設計流程涉及大量異質工具（合成、模擬、佈局、驗證等），各工具使用不同的輸入輸出格式、腳本語言和互動方式。根據 arXiv 論文《The Dawn of Agentic EDA》的分析，當前 EDA 工具被設計為人類互動介面，而非 Agent 可程式化呼叫的 API，造成以下問題：

- 缺乏標準化的 LLM-EDA 工具互動協定，必須為每個工具建立專屬介面
- 傳統 EDA 工具輸出非結構化的 log 或不透明的二進位資料庫，Agent 難以解析
- 商業 EDA 工具的專有格式（如 OpenAccess DB、專有 netlist 格式）缺乏開放文件

2.2 新興解決方案

Model Context Protocol (MCP)：Anthropic 提出的開放標準，已有 MCP4EDA 專案將其應用於開源 EDA 工具的 RTL-to-GDSII 流程。

AutoEDA：基於 MCP 的微服務架構，透過自然語言控制 RTL-to-GDSII，比傳統方法提升 9.9 倍準確率。

Siemens Fuse EDA AI Agent：商業方案，包含專用 RAG pipeline、多模態 EDA 資料庫、EDA 檔案格式專用解析器。

2.3 待辦事項

編號	任務	優先級	說明
T-1	為每個 EDA 工具建立 MCP Server 封裝	高	將 Yosys、Ngspice、OpenROAD 等工具的 CLI 封裝為 MCP Tool
T-2	定義統一的 EDA 資料交換格式	高	包含 netlist、constraint、timing report 等結構化格式
T-3	開發 EDA log 結構化解析器	中	將非結構化的工具輸出轉換為 Agent 可理解的格式
T-4	建立工具版本相容性矩陣	低	追蹤各開源工具版本間的相容性

3. LLM 的技術限制與可靠性

3.1 幻覺與程式碼生成問題

硬體設計對錯誤零容忍。根據多篇研究，LLM 在 Verilog 程式碼生成上存在顯著問題：

- 語法錯誤：LLM 生成的 Verilog 經常含有語法問題，尤其在複雜電路（如計數器、狀態機）
- 語意錯誤：商業 LLM 產生的 assertion 有 63% 的語意錯誤率
- 幻覺問題：LLM 可能生成看似合理但實際不存在的電路結構或 API 呼叫

3.2 推理與擴展性挑戰

- 現代 SoC 包含數十億節點，傳統 GNN 會遇到記憶體瓶頸與過度平滑問題
- 設計規格通常以曖昧的自然語言描述，LLM 難以完全解析為精確的形式化規格
- 開源模型（如 Llama3）在 EDA 任務上的表現顯著落後於閉源模型（如 GPT-4）

3.3 待辦事項

編號	任務	優先級	說明
T-5	建立 EDA 專用 LLM 微調流程	高	基於 ChatEDA 的 QLoRA 微調經驗，建立領域專用模型
T-6	開發 Verilog/VHDL 語法檢查器整合	高	在 Agent 輸出後自動進行語法檢查與修正
T-7	建立 prompt engineering 最佳實踐庫	中	針對各設計階段的專用 prompt 範本
T-8	評估開源 vs 閉源 LLM 在 EDA 任務的效能	中	比較 GPT-4、Llama、Mistral 等在合成/驗證任務的表現

4. 設計資料與訓練數據

4.1 數據稀缺的惡性循環

與軟體開發領域相比，IC 設計面臨嚴重的訓練數據不足問題：

- 高品質的工業級設計資料（RTL、netlist、GDSII）大多為專有且保密，無法用於訓練
- 開源 PDK（如 SKY130）僅限 130nm，與先進製程（7nm/5nm/3nm）差距極大
- 合成數據生成雖有幫助，但與真實工業設計的複雜度差距仍大

4.2 多模態資料處理

IC 設計涉及多種資料模態：RTL 程式碼、門級 netlist、實體佈局、時序報告、功耗分析等。如何讓 AI Agent 同時理解並處理這些異質資料，是一個重大的開放問題。

4.3 待辦事項

編號	任務	優先級	說明
T-9	建立開源 IC 設計資料集	高	彙整 SKY130/GF180 相關的開源設計範例與 benchmark
T-10	建立多模態資料前處理 pipeline	高	將 RTL、netlist、GDS、timing report 統一轉換為 LLM 可理解的格式
T-11	探索 Federated Learning 方案	中	讓多個組織協作訓練模型而不分享原始設計資料
T-12	建立合成數據生成框架	低	自動產生多樣化的設計範例用於訓練

5. 驗證與正確性保證

5.1 零容忍需求

IC 設計與軟體開發的最大差別在於：流片後無法修改。任何功能性錯誤、時序違規、DRC 問題都可能導致數百萬美元的損失。因此 AI Agent 生成的設計必須經過嚴格驗證。

5.2 當前挑戰

- LLM 生成的 testbench 覆蓋率不足，難以自動產生全面的驗證環境
- 形式化驗證（Formal Verification）需要專業知識，LLM 目前難以自主撰寫有效的 property/assertion
- 缺乏端到端 PPA（Power, Performance, Area）的標準化 benchmark
- 多次迭代的 feedback loop 效率低下，尚未實現 Correctness-by-Construction
- ECO 流程中的正確性驗證特別困難，因為只能修改局部邏輯而不能全面重新合成

5.3 待辦事項

編號	任務	優先級	說明
T-13	建立自動化 DRC/LVS 檢查流程	高	Agent 每次產出設計後自動執行 DRC/LVS 並解析結果
T-14	整合 Formal Verification 工具	高	探索將 SymbiYosys 等開源 FV 工具整合至 Agent 流程
T-15	建立端到端 PPA Benchmark	中	定義標準化的評估指標以比較 Agent 與人工設計的品質
T-16	開發自動 testbench 生成模組	中	結合 LLM 與 coverage-driven 策略自動產生測試

6. 安全性與 IP 保護

6.1 核心風險

半導體設計 IP 是公司最核心的資產。將 AI Agent 引入設計流程帶來新的安全挑戰：

- 依賴外部 API 服務存在 IP 洩漏風險，專有設計資料可能經由雲端暴露
- 自主 Agent 可能成為隱寫載體，微妙地洩漏隱私敏感的設計資訊
- Agent 生成的設計可能被植入硬體木馬（Hardware Trojan），傳統特徵比對方法難以偵測

6.2 待辦事項

編號	任務	優先級	說明
T-17	建立 Local LLM 部署環境	高	使用本地 GPU 執行推論，避免設計資料外傳
T-18	建立存取控制與稽核機制	高	定義 Agent 可存取的檔案、工具、網路範圍，並記錄所有操作
T-19	開發硬體木馬偵測機制	中	整合意圖分析等新型偵測方法
T-20	建立 IP 洩漏防護流程	中	監控 Agent 的輸入輸出，確保專有資訊不外洩

7. 基礎設施與運算資源

7.1 運算需求

AI Agent 系統需要同時滿足 LLM 推論和 EDA 工具執行的運算需求，這對基礎設施提出了更高要求：

- LLM 推論需要 GPU（如 NVIDIA A100/H100），EDA 工具執行主要依賴 CPU 與記憶體
- 大型設計的模擬可能需要數小時到數天，Agent 必須能管理長時間執行的任務
- 儲存需求大：單一 SoC 設計可能產生數 TB 的資料

7.2 待辦事項

編號	任務	優先級	說明
T-21	規劃 GPU + CPU 混合叢集架構	高	GPU 用於 LLM 推論，CPU 叢集用於 EDA 任務執行
T-22	建立非同步任務管理框架	高	Agent 能提交長時間 EDA job 並非同步等待結果
T-23	整合 Slurm 與 Agent 排程系統	中	讓 Agent 能自主提交和監控 HPC Job
T-24	建立分散式儲存方案	中	結合 BeeGFS/Lustre 管理大量設計資料

8. ECO Friendly Design（工程變更友善設計）

8.1 什麼是 ECO Friendly Design

ECO（Engineering Change Order）是 IC 設計後期常見的工程變更流程，當設計已進入後端佈局甚至流片階段時，若發現 bug 或需要功能調整，就需要透過 ECO 直接修改 gate-level netlist 而非重新走完整個合成流程。所謂「ECO Friendly Design」是指在設計初期就考慮後續 ECO 的可行性，讓設計更容易進行後期修改。

根據 Synopsys 的定義，Functional ECO 是直接對後合成版本的設計進行 patch 或修改，包含修復 RTL 錯誤、套用最佳化、或依新需求更新設計。這是一個極度耗時且容易出錯的人工流程。

8.2 AI Agent 在 ECO 流程中的挑戰

- 最小化修改範圍：Agent 必須能判斷 RTL 變更影響的最小 netlist 區域，避免不必要的重新合成
- Spare Cell 管理：ECO 修改只能使用佈局中已預留的 spare gate（備用單元），Agent 需理解可用的 spare cell 類型與位置
- Metal-Only ECO：後期 ECO 通常只能修改金屬層（metal layer），Agent 必須在此約束下將新邏輯對應到 spare gate 類型
- 繼承原有時序與布線：ECO 修改不能破壞原有的 timing closure 和 routing congestion，Agent 需具備後端實體設計的認知
- Formal Equivalence Checking：每次 ECO 修改後必須驗證修改後的 netlist 與新 RTL 功能等價，這對 LLM 是極大挑戰
- 經驗知識依賴：傳統 ECO 高度依賴資深工程師的經驗判斷，LLM 缺乏這種深層設計直覺

8.3 新興解決方案

NanDigits GOF AI：結合自然語言處理、智慧 Agent 與機器學習，自動化複雜 ECO 任務與 non-equivalence debug。

Metal Configurable Gate Array：專為 Metal-Only ECO 開發的 gate array spare cell，在 P&R 階段分散佈置於設計中，提升 ECO 彈性。

GNN 基礎的 Netlist 分析：利用圖神經網路自動識別 flattened netlist 中的模組邊界與功能分類，協助 Agent 理解修改影響範圍。

8.4 待辦事項

編號	任務	優先級	說明
T-28	建立 ECO 專用 Agent 流程	高	包含 RTL diff 分析、最小化重新合成、spare cell mapping
T-29	開發 Spare Cell 感知模組	高	Agent 能識別可用 spare gate 的類型、位置與數量

T-30	整合 Formal Equivalence Checking	高	ECO 修改後自動執行等價性驗證（如 SymbiYosys）
T-31	建立 ECO 影響分析引擎	中	自動評估 ECO 對 timing、area、power 的影響
T-32	開發 Metal-Only ECO 約束檢查器	中	確保 Agent 產生的 ECO patch 符合僅修改金屬層的限制

9. 人機協作與組織變革

8.1 人在迴圈的必要性

當前階段的 AI Agent 尚無法完全取代人類 IC 設計師。根據 IndexBox 的分析，驗證任務就佔了整個開發週期的 60%-70%，而這些任務往往需要豐富的經驗判斷。合理的方法是將 Agent 定位為「副駕駛」而非「自動駕駛」。

9.2 待辦事項

編號	任務	優先級	說明
T-25	定義 Human-in-the-Loop 檢查點	高	在關鍵決策點（如 floorplan、signoff、ECO）設置必要的人工審查
T-26	設計 Agent 操作看板 / 監控介面	中	讓設計師能即時查看 Agent 的決策、進度和結果
T-27	建立設計師培訓計畫	低	培訓團隊如何有效使用 AI Agent 並審查其輸出

10. 待辦事項總表

以下彙總所有待辦事項，依優先級分類：

10.1 高優先級（建議優先執行）

編號	任務	所屬領域
T-1	為每個 EDA 工具建立 MCP Server 封裝	工具整合
T-2	定義統一的 EDA 資料交換格式	工具整合
T-5	建立 EDA 專用 LLM 微調流程	LLM 可靠性
T-6	開發 Verilog/VHDL 語法檢查器整合	LLM 可靠性
T-9	建立開源 IC 設計資料集	設計資料
T-10	建立多模態資料前處理 pipeline	設計資料
T-13	建立自動化 DRC/LVS 檢查流程	驗證
T-14	整合 Formal Verification 工具	驗證
T-17	建立 Local LLM 部署環境	安全性
T-18	建立存取控制與稽核機制	安全性
T-21	規劃 GPU + CPU 混合叢集架構	基礎設施
T-22	建立非同步任務管理框架	基礎設施
T-25	定義 Human-in-the-Loop 檢查點	人機協作
T-28	建立 ECO 專用 Agent 流程	ECO Friendly Design
T-29	開發 Spare Cell 感知模組	ECO Friendly Design
T-30	整合 Formal Equivalence Checking	ECO Friendly Design

10.2 中優先級

編號	任務	所屬領域
T-3	開發 EDA log 結構化解析器	工具整合
T-7	建立 prompt engineering 最佳實踐庫	LLM 可靠性
T-8	評估開源 vs 閉源 LLM 在 EDA 任務的效能	LLM 可靠性

T-11	探索 Federated Learning 方案	設計資料
T-15	建立端到端 PPA Benchmark	驗證
T-16	開發自動 testbench 生成模組	驗證
T-19	開發硬體木馬偵測機制	安全性
T-20	建立 IP 洩漏防護流程	安全性
T-23	整合 Slurm 與 Agent 排程系統	基礎設施
T-24	建立分散式儲存方案	基礎設施
T-26	設計 Agent 操作今板 / 監控介面	人機協作
T-31	建立 ECO 影響分析引擎	ECO Friendly Design
T-32	開發 Metal-Only ECO 約束檢查器	ECO Friendly Design

10.3 低優先級

編號	任務	所屬領域
T-4	建立工具版本相容性矩陣	工具整合
T-12	建立合成數據生成框架	設計資料
T-27	建立設計師培訓計畫	人機協作

11. 參考資料

1. arXiv — [The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design](#)
2. arXiv — [AutoEDA: Enabling EDA Flow Automation through Microservice-Based LLM Agents](#)
3. arXiv — [A Survey of Research in Large Language Models for Electronic Design Automation](#)
4. arXiv — [Revolution or Hype? Seeking the Limits of Large Models in Hardware Design](#)
5. Oxford Academic — [Fully Automated Processor Chip Design: Motivation, Challenges and Future Directions](#)
6. Siemens — [Siemens launches Fuse EDA AI Agent](#)
7. Semiwiki — [EDA AI Agents Will Come in Three Waves](#)
8. IndexBox — [Human-Centered Agentic AI in EDA](#)
9. GitHub — [MCP4EDA: LLM-Powered MCP RTL-to-GDSII Automation](#)
10. GitHub — [Awesome-LLM4EDA](#)
11. SIGARCH — [AI in Chip Design: From Basic Tools to LLMs and AI Agents](#)
12. Semi Engineering — [LLM Technology For Chip Design](#)
13. Electronics FYI — [Agentic AI Raises Security Challenges in Chip Design](#)
14. Synopsys — [What is Functional ECO \(Engineering Change Order\)?](#)
15. NanDigits — [Functional Netlist ECO & GOF AI Platform](#)
16. NanDigits — [Metal Configurable Gate Array in Metal-Only ECO](#)
17. VLSI Pro — [ECO - Engineering Change Order](#)